K.R. Mangalam University

School of Engineering & Technology

Operating System Lab (ENCA252)

Lab Assignment 1

Implementation and Analysis of CPU Scheduling Algorithms (FCFS & SJF)

Submitted by:

Name: kshitij Marwah

Roll Number: 2401201021

Course: BCA (AI & DS)

Submitted To:

Dr. Jyoti Yadav

1. Objective

To implement and analyze CPU Scheduling Algorithms – **First Come First Serve (FCFS)** and **Shortest Job First (SJF)** — and evaluate their performance using standard metrics such as Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT).

2. Tools & Technologies Used

Tool / Technology	**Purpose**
Python 3.x	Used to implement FCFS and SJF scheduling
Built-in Libraries	Used for input handling and calculations
Terminal (Linux/Windows)	Used to execute the program and view output

3. Key Concepts

Term	**Formula**	**Description**
Completion Time (CT)	—	Time at which a process completes execution
Turnaround Time (TAT)	CT − AT	Total time from arrival to completion
WT	TAT − BT	Time spent waiting in the ready queue

4. Task 1 — Process Creation & Input Handling

Objective:

To understand how processes are represented in an operating system and how input data is structured.

Each process is created with attributes:

- Process ID (PID)
- Arrival Time (AT)
- Burst Time (BT)

The user inputs process details, which are stored in a list and displayed in a table.

Input Table

PID	Arrival Time (AT)	Burst Time (BT)
P1	4	6
P2	3	6
P3	7	5
P4	8	5

Screenshot — Input Table

5. Task 2 — FCFS Scheduling

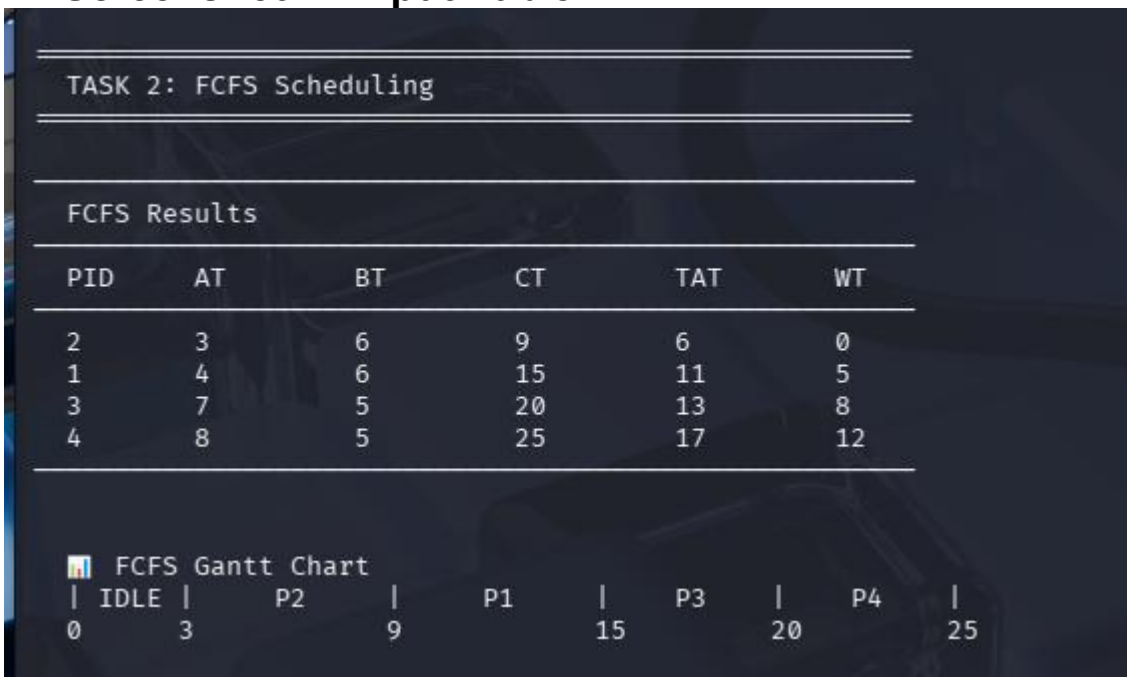
Objective:

To implement First Come First Serve (FCFS) scheduling algorithm.
Processes are executed in the order of arrival time.

FCFS Results Table

PID	AT	BT	CT	TAT	WT
P1	3	6	9	6	0
P2	4	6	15	11	5
P3	7	5	20	13	8
P4	8	5	25	17	12

Screenshot — Input Table



6. Task 3 — SJF Scheduling (Non-Preemptive)

Objective:

To implement Shortest Job First scheduling.
The process with the smallest burst time among arrived processes is selected first.

- Process ID (PID)
- Arrival Time (AT)
- Burst Time (BT)

SJF Results Table

PID	AT	BT	CT	TAT	WT
P2	3	6	9	6	0
P3	7	5	14	7	2
P4	8	5	19	11	6
P1	4	6	25	21	15

Screenshot — Input Table

7. Task 4 — Gantt Chart Representation

Objective:

To visualize process execution sequence.

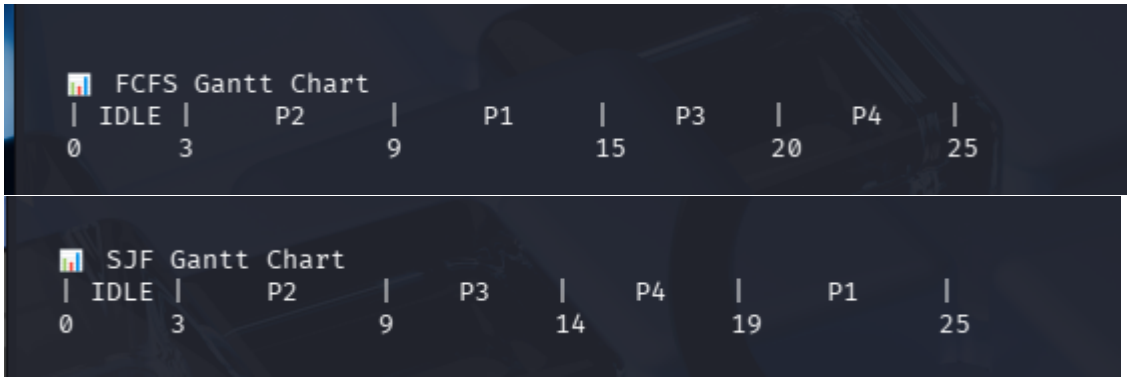
FCFS Gantt Chart

| IDLE | P2 | P1 | P3 | P4 |
0 3 9 15 20 25

SJF Gantt Chart

| IDLE | P2 | P3 | P4 | P1 |
0 3 9 14 19 25

Screenshot — Input Table



8. Task 5 — Performance Analysis & Comparison

Metric	FCFS	SJF
Average Turnaround Time (TAT)	11.75	11.25
Average Waiting Time (WT)	6.25	5.75

Analysis

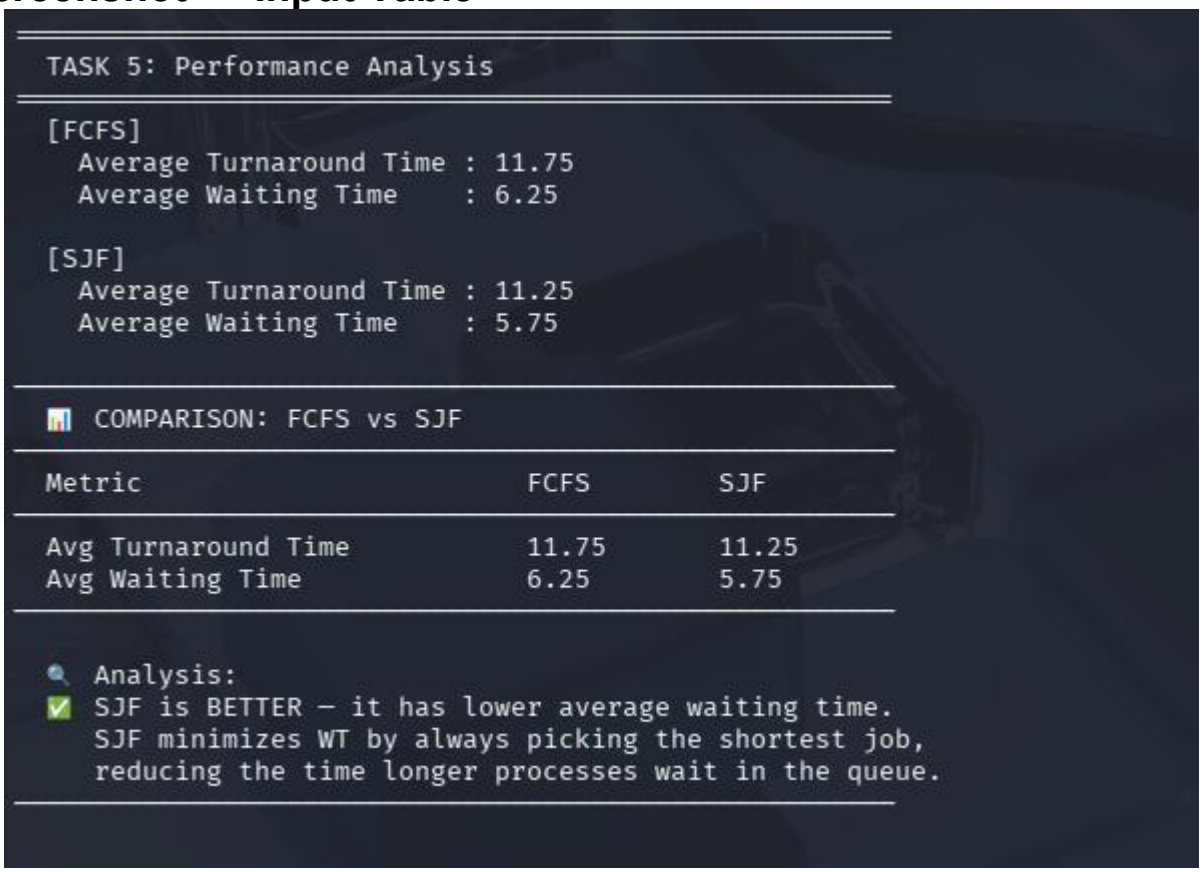
- SJF produces lower average waiting time
- FCFS is simple but may cause delay (convoy effect)
- SJF improves efficiency by executing shorter jobs first

Conclusion

SJF is better in this case because:

- It minimizes waiting time
- It improves CPU utilization
  However, it requires prior knowledge of burst times.

Screenshot — Input Table



9. GitHub Repository

Link – Github

10. Source Code

```
        width = max(len(label), (end - start) * 2)
        print(f"|{label.center(width)}", end="")
    print("|")

    # Time labels
    print("  ", end="")
    for (pid, start, end) in gantt:
        label = f" P{pid} " if pid != "IDLE" else " IDLE"
        width = max(len(label), (end - start) * 2)
        print(f"{str(start):<{width+1}}", end="")
    # Print last end time
    print(gantt[-1][2])
    print()

# ─────────────────────────────────────────
# TASK 5: Performance Analysis
# ─────────────────────────────────────────

def performance(processes, label):
    """Calculate and display average WT and TAT."""
    avg_tat = sum(p.tat for p in processes) / len(processes)
    avg_wt  = sum(p.wt  for p in processes) / len(processes)
    print(f"  [{label}]")
    print(f"    Average Turnaround Time : {avg_tat:.2f}")
    print(f"    Average Waiting Time    : {avg_wt:.2f}")
    return avg_tat, avg_wt

def compare(fcfs_tat, fcfs_wt, sjf_tat, sjf_wt):
    """Compare FCFS vs SJF and recommend."""
    print(f"\n{'─'*55}")
    print("  📊 COMPARISON: FCFS vs SJF")
    print(f"{'─'*55}")
    print(f"  {'Metric':<30}{'FCFS':<12}{'SJF'}")
    print(f"{'─'*55}")
    print(f"  {'Avg Turnaround Time':<30}{fcfs_tat:<12.2f}{sjf_tat:.2f}")
    print(f"  {'Avg Waiting Time':<30}{fcfs_wt:<12.2f}{sjf_wt:.2f}")
    print(f"{'─'*55}")

    print("\n  🔍 Analysis:")
    if sjf_wt < fcfs_wt:
        print("  ✅ SJF is BETTER — it has lower average waiting time.")
        print("     SJF minimizes WT by always picking the shortest job,")
        print("     reducing the time longer processes wait in the queue.")
    else:
```

Ln 21, Col 23 Spaces: 4 UTF-8 CRLF { } Python 3.14.2 Python 3.14 (64-bit)

```
            if not done[i] and p.at <= current_time
        ]

        # If no process available, jump to earliest arrival
        if not ready:
            next_arrival = min(p.at for i, p in enumerate(procs) if not done[i])
            gantt.append(("IDLE", current_time, next_arrival))
            current_time = next_arrival
            ready = [p for i, p in enumerate(procs) if not done[i] and p.at <= current_time]

        # Pick shortest burst time
        shortest = min(ready, key=lambda p: p.bt)

        start = current_time
        current_time += shortest.bt
        shortest.ct  = current_time
        shortest.tat = shortest.ct - shortest.at
        shortest.wt  = shortest.tat - shortest.bt

        gantt.append((shortest.pid, start, shortest.ct))

        # Mark as done
        for i, p in enumerate(procs):
            if p.pid == shortest.pid and not done[i]:
                done[i] = True
                break

        completed.append(shortest)

    return completed, gantt

# ─────────────────────────────────────────
# TASK 4: Gantt Chart (Text-based)
# ─────────────────────────────────────────

def draw_gantt(gantt, title="Gantt Chart"):
    """Draw a simple text-based Gantt chart."""
    print(f"\n  📊 {title}")
    print("  ", end="")

    # Top bar
    for (pid, start, end) in gantt:
        label = f" P{pid} " if pid != "IDLE" else " IDLE"
        width = max(len(label), (end - start) * 2)
```

Ln 21, Col 23 Spaces: 4 UTF-8 CRLF { } Python 3.14.2 Python 3.14 (64-bit)

```python
    """First Come First Serve — Non-Preemptive."""
    import copy
    procs = copy.deepcopy(processes)

    # Sort by Arrival Time
    procs.sort(key=lambda p: p.at)

    current_time = 0
    gantt = []   # stores (pid, start, end)

    for p in procs:
        # Handle CPU idle (no process has arrived yet)
        if current_time < p.at:
            gantt.append(("IDLE", current_time, p.at))
            current_time = p.at

        start = current_time
        current_time += p.bt          # process runs to completion
        p.ct  = current_time
        p.tat = p.ct - p.at
        p.wt  = p.tat - p.bt
        gantt.append((p.pid, start, p.ct))

    return procs, gantt

# ─────────────────────────────────────────
# TASK 3: SJF Scheduling (Non-Preemptive)
# ─────────────────────────────────────────

def sjf(processes):
    """Shortest Job First — Non-Preemptive."""
    import copy
    procs     = copy.deepcopy(processes)
    completed = []
    gantt     = []
    done      = [False] * len(procs)
    current_time = 0
    n = len(procs)

    for _ in range(n):
        # Build ready queue: arrived AND not done
        ready = [
            p for i, p in enumerate(procs)
            if not done[i] and p.at <= current_time
```

Ln 21, Col 23 Spaces: 4 UTF-8 CRLF { } Python 3.14.2 Python 3.14 (64-bit)

scheduling.py.py

scheduling.py.py > get_input

```python
import os

# ─────────────────────────────────────
# TASK 1: Process Class & Input Handling
# ─────────────────────────────────────

class Process:
    def __init__(self, pid, at, bt):
        self.pid = pid       # Process ID
        self.at  = at        # Arrival Time
        self.bt  = bt        # Burst Time
        self.ct  = 0         # Completion Time
        self.tat = 0         # Turnaround Time
        self.wt  = 0         # Waiting Time

def get_input():
    """Accept process details from the user."""
    processes = []
    n = int(input("Enter number of processes (4-5): "))
    print()
    for i in range(n):
        pid = int(input(f"  Process {i+1} — PID       : "))
        at  = int(input(f"  Process {i+1} — Arrival Time : "))
        bt  = int(input(f"  Process {i+1} — Burst Time   : "))
        processes.append(Process(pid, at, bt))
        print()
    return processes

def display_table(processes, title="Process Table"):
    """Display process details in a formatted table."""
    print(f"\n{'─'*55}")
    print(f"  {title}")
    print(f"{'─'*55}")
    print(f"  {'PID':<8}{'AT':<10}{'BT':<10}{'CT':<10}{'TAT':<10}{'WT'}")
    print(f"{'─'*55}")
    for p in processes:
        print(f"  {p.pid:<8}{p.at:<10}{p.bt:<10}{p.ct:<10}{p.tat:<10}{p.wt}")
    print(f"{'─'*55}\n")

# ─────────────────────────────────────
# TASK 2: FCFS Scheduling
# ─────────────────────────────────────

def fcfs(processes):
    """First Come First Serve — Non-Preemptive."""
```

Ln 21, Col 23 Spaces: 4 UTF-8 CRLF { } Python 3.14.2 Python 3.14 (64-bit)

```
    else:
        print("  ✅ FCFS performed equally well or better for this input.")
    print(f"{'~'*55}\n")

# ─────────────────────────────────────
# MAIN
# ─────────────────────────────────────

def main():
    os.system('clear')  # clears terminal on Linux
    print("=" * 55)
    print("   OS Lab Assignment 1 — CPU Scheduling")
    print("   FCFS & SJF (Non-Preemptive)")
    print("=" * 55)

    # Task 1 — Input
    processes = get_input()
    display_table(processes, "INPUT: Process Table")

    # Task 2 — FCFS
    print("\n" + "="*55)
    print("  TASK 2: FCFS Scheduling")
    print("="*55)
    fcfs_result, fcfs_gantt = fcfs(processes)
    display_table(fcfs_result, "FCFS Results")

    # Task 4 — FCFS Gantt
    draw_gantt(fcfs_gantt, "FCFS Gantt Chart")

    # Task 3 — SJF
    print("="*55)
    print("  TASK 3: SJF Scheduling (Non-Preemptive)")
    print("="*55)
    sjf_result, sjf_gantt = sjf(processes)
    display_table(sjf_result, "SJF Results")

    # Task 4 — SJF Gantt
    draw_gantt(sjf_gantt, "SJF Gantt Chart")

    # Task 5 — Performance
    print("="*55)
    print("  TASK 5: Performance Analysis")
    print("="*55)
    fcfs_tat, fcfs_wt = performance(fcfs_result, "FCFS")
    print()
    sjf_tat,  sjf_wt  = performance(sjf_result,  "SJF")
    compare(fcfs_tat, fcfs_wt, sjf_tat, sjf_wt)

if __name__ == "__main__":
    main()
```